

Mini Exercise: Unit Testing & Dependency Injection

In this exercise, you will learn how to make UI tests fast and reliable using dependency injection and a mock database.

The Scenario & Motivation

Imagine you have just been hired as a developer on a large collaborative project: the `InventoryManager`. The team is constantly complaining that running the UI tests takes forever because they connect to a real, live database. As you remember from your Unit Tests Lab, relying on real external services during testing is often **unnecessary** (we just need to test our code's behavior, not the database), incredibly **slow** (waiting for internet or disk responses), and potentially **costly**.

Thankfully, you know the perfect solution: **Mocking**. By creating a "fake" database that returns instantaneous, hard-coded responses, you can test the application efficiently without actually calling the real service.

However, because you are working on a massive codebase shared with many other developers, you must keep your footprint as small as possible to avoid merge conflicts and unintended bugs. Therefore, your task is to introduce this mock testing infrastructure while making the **minimal possible changes** to the existing core files.

To complete this task, follow these steps:

Running the Environment

Before modifying the code, familiarize yourself with running the application and the tests locally.

- **Run the app:** Open your terminal and run:

```
python app.py
```

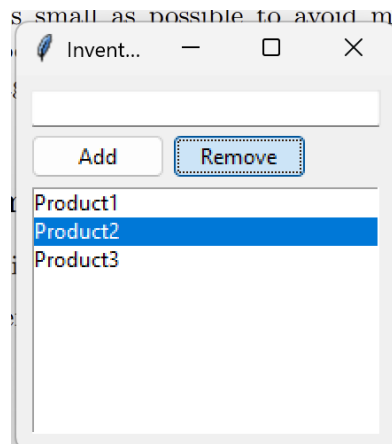


Figure 1: The application running in your terminal. You can add and remove products from the list. Notice the "waiting for response from database" message while the real slow database responds.

- **Run the tests:** Open your terminal and run:

```
python -m unittest tests/test_app.py tests/test_app_new.py
```

```

C:\Users\eladk\OneDrive - Technion\Documents\SEMESTERS\תוכנית הנדסה תעשייתית\03_Testing_MINI_3\MINIHW_3 - NEED FIX\di
st\assignment> python -m unittest tests/test_app.py tests/test_app_new.py
Establishing secure tunnel to HeavyDB... Please wait...
Establishing secure tunnel to HeavyDB... Please wait...
Establishing secure tunnel to HeavyDB... Please wait...
...
-----
Ran 5 tests in 24.237s
OK

```

Figure 2: The tests running in your terminal. Note: the last two tests are placeholders and must be replaced with meaningful tests.

Refactoring for Dependency Injection

You need to adjust the existing codebase to allow a database to be passed in, rather than hard-coded.

- **Update app.py:** Modify the code to support dependency injection by adding a `db` parameter.
- **Update tests/test_app.py:** Update the existing test file to inject your mock database instead of the real one.

Mock Implementation & New Tests

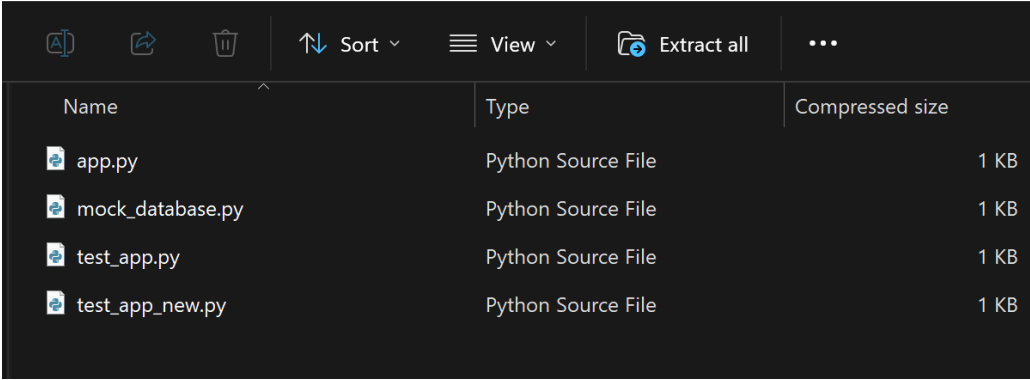
Once dependency injection is set up, you will implement the mock and use it.

- **Implement mock_database.py:** Create the logic for your mock database so it mimics the required behavior of the real database.
- **Update test_app_new.py:** Replace the provided placeholders with two meaningful, fast-running tests that utilize your newly injected mock database.

Submission Requirements

You should submit the following files:

1. A ZIP file named `ID1_ID2_ID3.zip` containing **exactly** these four files at the root of the ZIP (do not place them inside an inner folder):
 - `app.py`
 - `mock_database.py`
 - `test_app.py`
 - `test_app_new.py`



Name	Type	Compressed size
 app.py	Python Source File	1 KB
 mock_database.py	Python Source File	1 KB
 test_app.py	Python Source File	1 KB
 test_app_new.py	Python Source File	1 KB

Figure 3: How your ZIP file should look when opened. The files must be at the root level. The name of this ZIP should be in the format `ID1_ID2_ID3.zip`, where ID1, ID2, and ID3 are your three student IDs, or `ID1_ID2.zip` if you are only two students in the team.

Important Note

The goal of this exercise is for you to become comfortable with modifying existing code for testability while strictly adhering to constraints.

To simulate the requirement of making minimal changes to a shared codebase, there is a **strict line budget** for your refactoring:

- `app.py`: You may change **at most 5 lines**.
- `tests/test_app.py`: You may change **at most 5 lines**.

You are free to decide how to implement the mock database logic, provided the app behavior remains identical and the tests run successfully.

Testing your Submission

We have given you a local submission checker package containing `check_submission.py` and a `reference.zip` file. Put them in an empty folder with the ZIP you are about to submit, and run the checker script.

If it outputs a success message indicating your tests pass and line budgets are respected, then you have completed your assignment and can submit it with confidence.

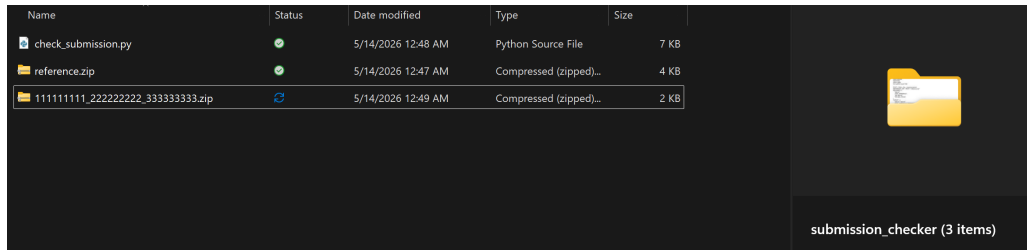


Figure 4: How to use `check_submission.py`. Put it in a folder with your submission zip and `reference.zip`, then run `python check_submission.py` in that directory.

The `check_submission.py` module does the following:

1. Unzips your submission into a temporary validation folder.
2. Checks if `app.py`, `mock_database.py`, `test_app.py`, and `test_app_new.py` are all present at the root level of the submission.
3. Compares your `app.py` and `test_app.py` against the `reference.zip` to ensure you have not exceeded the 5-line change budget for each file.
4. Runs the test suite locally to verify that your new mock database works and that all tests pass **under 2 seconds**.
5. Outputs a success message if all checks are passed. Your new tests will still be manually checked for quality.

```
C:\Users\eladk\OneDrive - Technion\Documents\SEMESTERS\תוכנית הנדסה תחילית\03_Testing_MINI_3\MINIHW_3 - NEED FIX\di
st\submission_checker>python check_submission.py
OK: auto-detected submission zip: 111111111_222222222_333333333.zip
OK: line budgets pass (app.py=2, tests/test_app.py=2)
OK: tests/test_app_new.py changed (content will be graded manually)
OK: mock template complete
OK: tests runtime: 0.25s
.....
Ran 5 tests in 0.252s
OK
OK: ran 5 tests
OK: all checks passed
```

Figure 5: Example of a successful output from the submission checker. Your new tests will still be manually checked for quality.